

Gymnázium Laury Bassi School Portal

Full Functional + Technical Project Spec

Source context: this spec is based on your executive summary and the current MVP UI screenshots. The project is a centralized school portal for attendance tracking, lesson logging, parent visibility, excuse submission, and admin management for Gymnázium Laury Bassi.

1. Product Definition

1.1 Product Name

Gymnázium Laury Bassi School Portal

1.2 Product Goal

Build a web application for Gymnázium Laury Bassi that gives the school one centralized system for:

- managing teachers
- managing parents
- managing students
- managing classes
- managing subjects
- managing class schedules
- recording daily attendance
- recording lesson logs
- allowing parents to view attendance
- allowing parents to submit absence excuses
- allowing teachers/admins to review excuses
- showing relevant notifications to users

1.3 Product Scope

The system has three first-class roles:

1. **Admin**
2. **Teacher**
3. **Parent**

The first version should be fully functional for these roles. It does not need Czech/Slovak translation yet. All interface text should be in English.

1.4 Required Infrastructure

Frontend: Next.js + React + TypeScript

Styling: Tailwind CSS

Backend: Supabase

Database: Supabase Postgres

Auth: Supabase Auth

Deployment: Vercel

Icons: lucide-react

Dates: date-fns or native Intl.DateTimeFormat

Optional but recommended:

Forms: react-hook-form

Validation: zod

Toasts: sonner or shadcn toast

UI primitives: shadcn/ui or custom Tailwind components

2. High-Level System Architecture

2.1 Architecture Overview

User Browser



Next.js App deployed on Vercel



Supabase Client / Server Actions / Route Handlers



Supabase Auth + Supabase Postgres + RLS

2.2 Core Responsibilities

Next.js

Responsible for:

- page routing
- role-based UI
- server-side session checks
- client interactions
- form submission
- calling Supabase
- rendering dashboards

Supabase

Responsible for:

- authentication
- user roles
- database storage
- row-level security
- relational data
- seed data
- backend policies

Vercel

Responsible for:

- hosting the web app
 - environment variables
 - production deployment
-

3. User Roles

3.1 Admin

Admins manage the system.

Admin accounts can be assigned manually through Supabase/database scripts. Once assigned, the admin should have a full web interface.

Admins can:

- add/edit/deactivate teachers
- add/edit/deactivate parents
- add/edit/deactivate students
- add/edit/deactivate classes
- add/edit/deactivate subjects
- create/edit/deactivate teacher assignments
- create/edit/deactivate class schedules
- link parents to students
- view all attendance records
- edit attendance records if needed
- view all lesson logs
- view all excuse requests
- approve/reject excuse requests
- view operational dashboard metrics

3.2 Teacher

Teachers manage records for their assigned classes/subjects.

Teachers can:

- view their assigned classes
- view today's scheduled class sessions
- record attendance for assigned classes
- mark students present/absent/late
- create lesson logs for assigned classes
- update existing lesson logs
- view excuse requests for students in their assigned classes
- approve/reject excuse requests

3.3 Parent

Parents view data for their linked children.

Parents can:

- view their child's attendance history
 - view absence summary
 - view unexcused/pending/excused statuses
 - submit excuses for absences
 - view notification feed
-

4. Auth and Role Routing

4.1 Login

Route:

/login

Fields:

Email

Password

Actions:

Sign in

Forgot password

Toggle password visibility

Behavior:

- User enters email/password
- Supabase Auth authenticates user
- App loads user profile from `profiles`
- App redirects based on role

Redirect rules:

admin → /admin/dashboard

teacher → /teacher/attendance

parent → /parent/notifications

4.2 Logout

Logout should:

- clear Supabase session
- redirect to `/login`

4.3 Route Protection

Every protected route must check:

1. user is authenticated
2. user has required role

Examples:

/admin/* → admin only
/teacher/* → teacher only
/parent/* → parent only

If unauthorized:

not logged in → /login
wrong role → /unauthorized

5. UI Language

All website UI should be in **English** for now.

Do not use Slovak/Czech UI labels in the first rebuild.

Examples:

Dochádzka → Attendance
Záznamy hodín → Lesson Logs
Prítomný → Present
Neprítomný → Absent
Odhlásiť → Log out
Prihláste sa → Sign in
Trieda → Class

Future localization can be added later.

6. Global UI Design

6.1 Visual Style

Match the current MVP visual direction:

Background: warm off-white / light gray

Cards: white, rounded, subtle border
Primary color: deep green
Text: black/dark gray
Secondary text: muted gray
Status badges: soft pill badges
Layout: centered max-width container

6.2 Shared Components

Create shared components:

AppHeader
RoleLayout
TabNavigation
PageContainer
Card
Button
Input
Textarea
Select
StatusBadge
AvatarInitials
EmptyState
LoadingState
ErrorState
ConfirmDialog
DataTable
SearchInput

6.3 Header

Header should include:

- school logo
 - school name: **Gymnázium Laury Bassi**
 - subtitle: **School Portal**
 - current user name
 - role label
 - avatar initials
 - logout button
-

7. Application Routes

7.1 Public Routes

/login
/forgot-password
/unauthorized

7.2 Admin Routes

/admin/dashboard
/admin/teachers
/admin/parents
/admin/students
/admin/classes
/admin/subjects
/admin/schedules
/admin/assignments
/admin/attendance
/admin/lessons
/admin/excuses

7.3 Teacher Routes

/teacher/attendance
/teacher/lessons
/teacher/excuses

7.4 Parent Routes

/parent/notifications
/parent/attendance
/parent/submit-excuse

8. Database Schema

Use Supabase Postgres.

8.1 profiles

Represents app-level user profiles tied to Supabase Auth users.

```
create table profiles (  
  id uuid primary key references auth.users(id) on delete cascade,  
  full_name text not null,  
  email text not null unique,  
  role text not null check (role in ('admin', 'teacher', 'parent')),  
  initials text,  
  phone text,  
  is_active boolean not null default true,  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

Notes:

- `id` must match `auth.users.id`
 - Role determines routing and permissions
 - Admin assignment can be done manually through SQL
-

8.2 classes

```
create table classes (  
  id uuid primary key default gen_random_uuid(),  
  name text not null,  
  school_year text,  
  is_active boolean not null default true,  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

Example:

3.A

8.3 students

```
create table students (  
  id uuid primary key default gen_random_uuid(),  
  full_name text not null,  
  initials text,  
  class_id uuid references classes(id),
```

```
is_active boolean not null default true,  
created_at timestampz not null default now(),  
updated_at timestampz not null default now()  
);
```

8.4 parent_students

Links parents to students.

```
create table parent_students (  
  parent_id uuid not null references profiles(id) on delete cascade,  
  student_id uuid not null references students(id) on delete cascade,  
  relationship text default 'parent',  
  created_at timestampz not null default now(),  
  primary key (parent_id, student_id)  
);
```

8.5 subjects

```
create table subjects (  
  id uuid primary key default gen_random_uuid(),  
  name text not null unique,  
  is_active boolean not null default true,  
  created_at timestampz not null default now(),  
  updated_at timestampz not null default now()  
);
```

Example:

Mathematics
English
History
Biology

8.6 teacher_assignments

Links teachers to classes and subjects.

```
create table teacher_assignments (  
  id uuid primary key default gen_random_uuid(),
```

```
teacher_id uuid not null references profiles(id) on delete cascade,  
class_id uuid not null references classes(id) on delete cascade,  
subject_id uuid not null references subjects(id) on delete cascade,  
is_active boolean not null default true,  
created_at timestamptz not null default now(),  
updated_at timestamptz not null default now(),  
unique (teacher_id, class_id, subject_id)  
);
```

Example:

Mgr. Jana Bielíková teaches Mathematics for Class 3.A

8.7 class_schedules

Defines recurring expected class sessions.

```
create table class_schedules (  
id uuid primary key default gen_random_uuid(),  
class_id uuid not null references classes(id) on delete cascade,  
subject_id uuid not null references subjects(id) on delete cascade,  
teacher_id uuid not null references profiles(id) on delete cascade,  
day_of_week int not null check (day_of_week between 0 and 6),  
start_time time not null,  
end_time time not null,  
room text,  
is_active boolean not null default true,  
created_at timestamptz not null default now(),  
updated_at timestamptz not null default now()  
);
```

Day mapping:

```
0 = Sunday  
1 = Monday  
2 = Tuesday  
3 = Wednesday  
4 = Thursday  
5 = Friday  
6 = Saturday
```

8.8 class_sessions

A concrete class occurrence for a date.

```
create table class_sessions (  
  id uuid primary key default gen_random_uuid(),  
  class_id uuid not null references classes(id) on delete cascade,  
  subject_id uuid not null references subjects(id) on delete cascade,  
  teacher_id uuid not null references profiles(id) on delete cascade,  
  schedule_id uuid references class_schedules(id) on delete set null,  
  session_date date not null,  
  start_time time,  
  end_time time,  
  status text not null default 'open' check (status in ('open', 'completed', 'cancelled')),  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now(),  
  unique (class_id, subject_id, teacher_id, session_date, start_time)  
);
```

Purpose:

- attendance records attach to this
 - lesson logs attach to this
 - excuses indirectly attach through attendance records
-

8.9 attendance_records

```
create table attendance_records (  
  id uuid primary key default gen_random_uuid(),  
  session_id uuid not null references class_sessions(id) on delete cascade,  
  student_id uuid not null references students(id) on delete cascade,  
  status text not null check (status in ('present', 'absent', 'late')),  
  recorded_by uuid references profiles(id),  
  recorded_at timestamptz not null default now(),  
  updated_at timestamptz not null default now(),  
  unique (session_id, student_id)  
);
```

8.10 lesson_logs

```
create table lesson_logs (  

```

```
id uuid primary key default gen_random_uuid(),
session_id uuid not null references class_sessions(id) on delete cascade,
teacher_id uuid not null references profiles(id) on delete cascade,
topic text,
content text not null,
homework text,
created_at timestamptz not null default now(),
updated_at timestamptz not null default now(),
unique (session_id)
);
```

8.11 excuse_requests

```
create table excuse_requests (
id uuid primary key default gen_random_uuid(),
attendance_record_id uuid not null references attendance_records(id) on delete cascade,
student_id uuid not null references students(id) on delete cascade,
parent_id uuid not null references profiles(id) on delete cascade,
reason_category text not null check (
    reason_category in ('medical', 'family', 'travel', 'other')
),
description text,
status text not null default 'pending' check (
    status in ('pending', 'approved', 'rejected')
),
reviewed_by uuid references profiles(id),
reviewed_at timestamptz,
created_at timestamptz not null default now(),
updated_at timestamptz not null default now(),
unique (attendance_record_id, parent_id)
);
```

8.12 notifications

```
create table notifications (
id uuid primary key default gen_random_uuid(),
recipient_id uuid not null references profiles(id) on delete cascade,
student_id uuid references students(id) on delete cascade,
type text not null check (
    type in (
        'absence_recorded',
        'excuse_submitted',

```

```
        'excuse_approved',
        'excuse_rejected',
        'system'
    )
),
title text,
message text not null,
status text not null default 'info' check (
    status in ('info', 'pending', 'success', 'error')
),
read_at timestamptz,
created_at timestamptz not null default now()
);
```

9. Important Data Rules

9.1 Attendance Display Logic

Attendance and excuse status are separate.

```
type AttendanceStatus = "present" | "absent" | "late";
type ExcuseStatus = "pending" | "approved" | "rejected";
```

Parent-facing display logic:

```
function getParentDisplayStatus(attendanceStatus, excuseStatus) {
    if (attendanceStatus === "present") return "Present";
    if (attendanceStatus === "late") return "Late";

    if (attendanceStatus === "absent") {
        if (!excuseStatus) return "Unexcused";
        if (excuseStatus === "pending") return "Pending";
        if (excuseStatus === "approved") return "Excused";
        if (excuseStatus === "rejected") return "Unexcused";
    }

    return "Unknown";
}
```

9.2 Absence Counts

Total absences:

attendance_records where status = absent

Unexcused absences:

attendance_records where status = absent

AND no approved/pending excuse exists

OR latest excuse is rejected

For MVP, if an absence has a pending excuse, do not count it as unexcused.

9.3 Duplicate Prevention

Prevent duplicates for:

profiles.email

subjects.name

teacher_assignments(teacher_id, class_id, subject_id)

parent_students(parent_id, student_id)

attendance_records(session_id, student_id)

lesson_logs(session_id)

excuse_requests(attendance_record_id, parent_id)

10. Supabase RLS Policies

Enable RLS on all app tables.

```
alter table profiles enable row level security;  
alter table classes enable row level security;  
alter table students enable row level security;  
alter table parent_students enable row level security;  
alter table subjects enable row level security;  
alter table teacher_assignments enable row level security;  
alter table class_schedules enable row level security;  
alter table class_sessions enable row level security;  
alter table attendance_records enable row level security;  
alter table lesson_logs enable row level security;  
alter table excuse_requests enable row level security;  
alter table notifications enable row level security;
```

10.1 Helper Function: Current Role

```
create or replace function public.current_user_role()
returns text
language sql
security definer
as $$
    select role from public.profiles where id = auth.uid()
$$;
```

10.2 Helper Function: Is Admin

```
create or replace function public.is_admin()
returns boolean
language sql
security definer
as $$
    select exists (
        select 1 from public.profiles
        where id = auth.uid()
        and role = 'admin'
        and is_active = true
    )
$$;
```

10.3 Policy Summary

Admin

Admin can read/write almost everything.

Admins can select, insert, update, delete:

- profiles
- classes
- students
- parent_students
- subjects
- teacher_assignments
- class_schedules
- class_sessions
- attendance_records
- lesson_logs
- excuse_requests
- notifications

For safer deletion, prefer soft delete with `is_active = false`.

Teacher

Teacher can:

- read own profile
- read assigned classes
- read assigned subjects
- read students in assigned classes
- read/create/update sessions for assigned classes
- read/create/update attendance for assigned classes
- read/create/update lesson logs for own sessions
- read/update excuse requests for assigned students/classes
- read own notifications

Parent

Parent can:

- read own profile
- read linked students
- read linked student class
- read linked student attendance
- read own submitted excuses
- create excuse for linked student absence
- read own notifications

11. Admin Portal Spec

11.1 Admin Layout

Route prefix:

/admin

Header:

Gymnázium Laury Bassi
School Portal
[Admin Name]
Admin

[Initials Avatar]

Log out

Navigation:

Dashboard

Teachers

Parents

Students

Classes

Subjects

Schedules

Assignments

Attendance

Lesson Logs

Excuses

11.2 /admin/dashboard

Purpose:

Operational overview.

Cards:

Total Students

Total Teachers

Total Parents

Total Classes

Today's Sessions

Pending Excuses

Lesson Logs Today

Absences Today

Sections:

Recent Absences

Recent Lesson Logs

Recent Excuse Requests

11.3 /admin/teachers

Admin can:

- view all teachers
- search teachers
- add teacher
- edit teacher
- deactivate teacher
- view assigned classes/subjects

Teacher fields:

Full Name

Email

Initials

Phone optional

Active Status

Actions:

Add Teacher

Edit

Deactivate

Reactivate

Important implementation note:

Creating a teacher requires creating a Supabase Auth user plus a **profiles** row. For MVP, you can either:

1. create users manually in Supabase and let admin edit profile data, or
2. implement a server-only admin creation endpoint using **SUPABASE_SERVICE_ROLE_KEY**.

Do not expose service role key to the client.

11.4 /admin/parents

Admin can:

- view all parents
- search parents
- add parent
- edit parent

- deactivate parent
- link parent to student
- unlink parent from student

Parent fields:

Full Name

Email

Initials

Phone optional

Linked Students

Active Status

11.5 /admin/students

Admin can:

- view all students
- search students
- filter by class
- add student
- edit student
- deactivate student
- assign student to class
- link student to parent

Student fields:

Full Name

Initials

Class

Linked Parents

Active Status

11.6 /admin/classes

Admin can:

- view classes
- add class
- edit class
- deactivate class

- view students in class

Class fields:

Name

School Year optional

Active Status

Example:

3.A

11.7 /admin/subjects

Admin can:

- view subjects
- add subject
- edit subject
- deactivate subject

Subject fields:

Name

Active Status

11.8 /admin/assignments

Admin can assign teachers to classes and subjects.

Fields:

Teacher

Class

Subject

Active Status

Actions:

Create Assignment

Edit Assignment

Deactivate Assignment

Example:

Mgr. Jana Bielíková → Mathematics → Class 3.A

11.9 /admin/schedules

Admin can create recurring schedules.

Fields:

Class
Subject
Teacher
Day of Week
Start Time
End Time
Room optional
Active Status

Actions:

Create Schedule
Edit Schedule
Deactivate Schedule

Purpose:

Schedules determine:

- teacher daily sessions
- valid lesson log options
- expected attendance sessions
- future reminder/missing record logic

11.10 /admin/attendance

Admin can:

- view all attendance records
- filter by date
- filter by class

- filter by student
- filter by teacher
- filter by status
- edit attendance status

Columns:

Date
Class
Subject
Student
Status
Recorded By
Recorded At
Excuse Status

11.11 /admin/lessons

Admin can:

- view lesson logs
- filter by date
- filter by class
- filter by subject
- filter by teacher
- open full lesson details
- edit/delete lesson logs if needed

Columns:

Date
Class
Subject
Teacher
Topic
Last Updated

11.12 /admin/excuses

Admin can:

- view all excuse requests

- filter by status
- filter by student
- filter by class
- approve excuse
- reject excuse

Columns:

Student
Class
Absence Date
Reason Category
Description
Submitted By
Status
Reviewed By
Reviewed At

On approve:

```
excuse_requests.status = approved  
reviewed_by = current admin  
reviewed_at = now()  
create notification for parent
```

On reject:

```
excuse_requests.status = rejected  
reviewed_by = current admin  
reviewed_at = now()  
create notification for parent
```

12. Teacher Portal Spec

12.1 Teacher Layout

Route prefix:

/teacher

Header:

Gymnázium Laury Bassi

School Portal

[Teacher Name]

Teacher

[Initials Avatar]

Log out

Navigation:

Attendance

Lesson Logs

Excuses

12.2 /teacher/attendance

Page title:

Today's Attendance — Class 3.A

April 14, 2026

Controls:

Class selector

Session selector if multiple scheduled sessions exist

Summary card:

Total: 5

Present: 5

Absent: 0

Late: 0

Student row:

[Initials] Anna Kováčová [Present] [Absent] [Late]

Behavior:

1. Load teacher assignments
2. Select default class/session
3. Get or create class session for today
4. Load student roster
5. Initialize attendance records if missing
6. Allow teacher to update status

7. Update counts instantly
8. Persist changes to Supabase

Status button behavior:

Present active → green

Absent active → red

Late active → yellow/orange

Inactive → outline/gray

Required backend functions:

```
getTeacherAssignments(teacherId)
getTodaySessionsForTeacher(teacherId)
getOrCreateClassSession(input)
initializeAttendanceRecords(sessionId, classId)
getAttendanceRecords(sessionId)
updateAttendanceRecord(recordId, status)
```

12.3 /teacher/lessons

Page title:

Lesson Logs

April 14, 2026

Form section:

Record Today's Lesson

Fields:

Class Session selector

Lesson Topic

Lesson Description

Homework optional

Session summary card:

Mathematics

Class 3.A

April 14, 2026

Actions:

Save Lesson Log
Update Lesson Log

Behavior:

- Teacher selects class session
- If a lesson log exists, load it
- If no lesson log exists, show empty form
- Save creates or updates `lesson_logs`
- A session may only have one lesson log

Required backend functions:

`getLessonLog(sessionId)`
`upsertLessonLog(input)`
`getTeacherLessonSessions(teacherId, date)`

12.4 /teacher/excuses

Purpose:

Let teachers review excuses for students in assigned classes.

Teacher can:

- view pending excuses
- view approved/rejected excuses
- approve excuse
- reject excuse

Filters:

Status
Class
Student
Date

Columns:

Student
Class
Absence Date
Reason Category
Description

Submitted By
Status
Actions

Actions:

Approve
Reject

Behavior:

- Teacher can only see excuses for students in classes they are assigned to
 - Approval/rejection creates parent notification
-

13. Parent Portal Spec

13.1 Parent Layout

Route prefix:

/parent

Header:

Gymnázium Laury Bassi
School Portal
[Parent Name]
Parent
[Initials Avatar]
Log out

Dashboard heading:

Parent Dashboard
Student: Anna Kováčová — Class 3.A

If parent has multiple linked students:

Student selector

Navigation:

13.2 /parent/notifications

Page title:

Recent Notifications

Notification row:

[Icon] Message	Date
----------------	------

Notification examples:

Excuse submitted for Anna Kováčová — Absence on March 7, 2026 (Medical). Awaiting review.
Excuse accepted for Anna Kováčová — Absence on March 3, 2026 (Medical).
Anna Kováčová was marked absent from Mathematics on March 12, 2026.

Notification types:

absence_recorded
excuse_submitted
excuse_approved
excuse_rejected
system

Behavior:

- Parent sees only own notifications
- Newest first
- Optional: clicking notification marks as read

13.3 /parent/attendance

Page title:

Attendance Record

Summary cards:

Total Absences: 3

Unexcused: 0

Attendance row:

April 7, 2026	Mathematics	Present
March 25, 2026	Mathematics	Present
March 12, 2026	Mathematics	Pending
March 10, 2026	Mathematics	Present

Status badges:

Present

Late

Pending

Excused

Unexcused

Behavior:

- Parent only sees linked student data
- Attendance list should join:
 - attendance_records
 - class_sessions
 - subjects
 - excuse_requests
- Display status uses logic from section 9

13.4 /parent/submit-excuse

Purpose:

Allow parent to submit an excuse for an absence.

Fields:

Student

Absence

Reason Category

Description

Reason categories:

Medical
Family
Travel
Other

Behavior:

- Only show absences for linked student
- Only show attendance records where status = absent
- Do not show absences that already have a pending or approved excuse
- Parent submits excuse
- System creates `excuse_requests`
- System creates parent notification confirming submission
- System may create teacher/admin notification

Validation:

Absence required
Reason category required
Description optional but recommended
Duplicate excuse blocked

14. Backend/Data Access Layer

Create data access functions under:

`lib/data/`

Recommended files:

`lib/data/profiles.ts`
`lib/data/admin.ts`
`lib/data/teachers.ts`
`lib/data/parents.ts`
`lib/data/students.ts`
`lib/data/classes.ts`
`lib/data/subjects.ts`
`lib/data/schedules.ts`
`lib/data/assignments.ts`
`lib/data/sessions.ts`
`lib/data/attendance.ts`
`lib/data/lessons.ts`
`lib/data/excuses.ts`

lib/data/notifications.ts

14.1 Auth Functions

signIn(email: string, password: string)
signOut()
getCurrentUser()
getCurrentProfile()
requireAuth()
requireRole(role: "admin" | "teacher" | "parent")

14.2 Admin Functions

getAdminDashboardStats()

listTeachers()
createTeacher(input)
updateTeacher(id, input)
deactivateTeacher(id)

listParents()
createParent(input)
updateParent(id, input)
deactivateParent(id)

listStudents(filters)
createStudent(input)
updateStudent(id, input)
deactivateStudent(id)

listClasses()
createClass(input)
updateClass(id, input)
deactivateClass(id)

listSubjects()
createSubject(input)
updateSubject(id, input)
deactivateSubject(id)

listTeacherAssignments()
createTeacherAssignment(input)
updateTeacherAssignment(id, input)
deactivateTeacherAssignment(id)

listClassSchedules()
createClassSchedule(input)
updateClassSchedule(id, input)
deactivateClassSchedule(id)

linkParentToStudent(parentId, studentId)
unlinkParentFromStudent(parentId, studentId)

14.3 Teacher Functions

getTeacherAssignments(teacherId)
getTeacherSchedulesForDate(teacherId, date)
getOrCreateClassSession(input)
getStudentsForClass(classId)
initializeAttendanceForSession(sessionId, classId)
getAttendanceForSession(sessionId)
updateAttendanceStatus(recordId, status)
getTeacherLessonSessions(teacherId, date)
getLessonLog(sessionId)
upsertLessonLog(input)
getExcusesForTeacher(teacherId, filters)
reviewExcuse(excuseId, reviewerId, status)

14.4 Parent Functions

getLinkedStudents(parentId)
getParentNotifications(parentId)
getStudentAttendanceForParent(parentId, studentId)
getAbsenceSummaryForParent(parentId, studentId)
getExcusableAbsences(parentId, studentId)
submitExcuse(input)

15. Notifications Behavior

15.1 When Attendance Is Marked Absent

When a student is marked absent:

- create parent notification for linked parents

Message:

Anna Kováčová was marked absent from Mathematics on March 12, 2026.

Avoid duplicate notifications for the same attendance record if status is repeatedly saved as absent.

15.2 When Excuse Is Submitted

Create parent notification:

Excuse submitted for Anna Kováčová — Absence on March 12, 2026 (Family). Awaiting review.

Optional:

Create teacher/admin notification:

New excuse submitted for Anna Kováčová.

15.3 When Excuse Is Approved

Create parent notification:

Excuse accepted for Anna Kováčová — Absence on March 12, 2026 (Family).

15.4 When Excuse Is Rejected

Create parent notification:

Excuse rejected for Anna Kováčová — Absence on March 12, 2026 (Family).

16. Seed Data

Create seed data for development/demo.

16.1 Demo Users

Admin

Name: Admin User

Email: admin@gybassi.cz

Password: GLBAdmin2026!

Role: admin

Initials: AU

Teacher

Name: Mgr. Jana Bieliková

Email: jana.bielikova@gybassi.cz

Password: GLBPortal2026!

Role: teacher

Initials: MJ

Parent

Name: Martina Kováčová

Email: parent@glb.sk

Password: GLBParent2026!

Role: parent

Initials: MK

16.2 Class

Class: 3.A

16.3 Subject

Subject: Mathematics

16.4 Students

Anna Kováčová

Jana Kučerová

Lucia Horváthová

Marek Novák

Peter Szabó

16.5 Relationships

Martina Kováčová → Anna Kováčová

Mgr. Jana Bieliková → Mathematics → Class 3.A

16.6 Demo Attendance History

April 7, 2026 Mathematics Present

March 25, 2026 Mathematics Present

March 12, 2026	Mathematics	Absent + Pending Excuse
March 10, 2026	Mathematics	Present
March 3, 2026	Mathematics	Absent + Approved Excuse

16.7 Demo Notifications

Excuse submitted for Anna Kováčová — Absence on March 7, 2026 (Medical). Awaiting review.
Excuse submitted for Anna Kováčová — Absence on March 12, 2026 (Family). Awaiting review.
Excuse accepted for Anna Kováčová — Absence on March 3, 2026 (Medical).

17. Project Structure

Recommended Next.js App Router structure:

```
app/  
  layout.tsx  
  page.tsx  
  
login/  
  page.tsx  
  
forgot-password/  
  page.tsx  
  
unauthorized/  
  page.tsx  
  
admin/  
  layout.tsx  
  dashboard/  
    page.tsx  
  teachers/  
    page.tsx  
  parents/  
    page.tsx  
  students/  
    page.tsx  
  classes/  
    page.tsx  
  subjects/  
    page.tsx  
  schedules/
```

- page.tsx
- assignments/
 - page.tsx
- attendance/
 - page.tsx
- lessons/
 - page.tsx
- excuses/
 - page.tsx

- teacher/
 - layout.tsx
 - attendance/
 - page.tsx
 - lessons/
 - page.tsx
 - excuses/
 - page.tsx

- parent/
 - layout.tsx
 - notifications/
 - page.tsx
 - attendance/
 - page.tsx
 - submit-excuse/
 - page.tsx

- components/
 - shared/
 - app-header.tsx
 - avatar-initials.tsx
 - button.tsx
 - card.tsx
 - data-table.tsx
 - empty-state.tsx
 - error-state.tsx
 - input.tsx
 - loading-state.tsx
 - page-container.tsx
 - select.tsx
 - status-badge.tsx
 - tab-navigation.tsx
 - textarea.tsx

admin/

- admin-nav.tsx
- admin-dashboard-cards.tsx
- teacher-form.tsx
- parent-form.tsx
- student-form.tsx
- class-form.tsx
- subject-form.tsx
- schedule-form.tsx
- assignment-form.tsx
- excuse-review-table.tsx

teacher/

- teacher-nav.tsx
- attendance-summary.tsx
- student-attendance-row.tsx
- lesson-log-form.tsx
- excuse-review-list.tsx

parent/

- parent-nav.tsx
- notification-list.tsx
- notification-row.tsx
- attendance-summary-cards.tsx
- attendance-record-list.tsx
- submit-excuse-form.tsx

lib/

- supabase/
 - client.ts
 - server.ts
 - admin.ts

auth/

- require-auth.ts
- require-role.ts
- redirects.ts

data/

- profiles.ts
- admin.ts
- teachers.ts
- parents.ts

students.ts
classes.ts
subjects.ts
schedules.ts
assignments.ts
sessions.ts
attendance.ts
lessons.ts
excuses.ts
notifications.ts

utils/
 dates.ts
 statuses.ts
 initials.ts

types/
 database.ts
 domain.ts

supabase/
 migrations/
 seed.sql

18. Environment Variables

In `.env.local`:

```
NEXT_PUBLIC_SUPABASE_URL=  
NEXT_PUBLIC_SUPABASE_ANON_KEY=  
SUPABASE_SERVICE_ROLE_KEY=
```

In Vercel:

```
NEXT_PUBLIC_SUPABASE_URL  
NEXT_PUBLIC_SUPABASE_ANON_KEY  
SUPABASE_SERVICE_ROLE_KEY
```

Rules:

- `NEXT_PUBLIC_SUPABASE_URL` can be exposed to browser
- `NEXT_PUBLIC_SUPABASE_ANON_KEY` can be exposed to browser

- `SUPABASE_SERVICE_ROLE_KEY` must never be used client-side
 - service role key only belongs in server-only code or seed/admin scripts
-

19. Acceptance Criteria

19.1 Auth

- user can sign in with email/password
- user is redirected based on role
- user can log out
- unauthenticated users cannot access protected pages
- wrong-role users cannot access protected pages

19.2 Admin

- admin can view dashboard stats
- admin can add/edit/deactivate teachers
- admin can add/edit/deactivate parents
- admin can add/edit/deactivate students
- admin can add/edit/deactivate classes
- admin can add/edit/deactivate subjects
- admin can create/edit/deactivate schedules
- admin can assign teachers to class/subject
- admin can link parents to students
- admin can view attendance
- admin can view lesson logs
- admin can approve/reject excuses

19.3 Teacher Attendance

- teacher sees only assigned classes
- teacher can select class/session
- student roster loads correctly
- attendance initializes if missing
- teacher can mark present/absent/late
- summary counts update
- records persist after refresh

19.4 Teacher Lesson Logs

- teacher sees only assigned class sessions
- teacher can create lesson log
- teacher can update existing lesson log
- duplicate lesson logs are prevented

19.5 Teacher Excuses

- teacher sees excuses for assigned classes only
- teacher can approve/reject pending excuses
- parent receives notification after review

19.6 Parent Notifications

- parent sees own notifications only
- notifications are ordered newest first
- notification message/date/status render correctly

19.7 Parent Attendance

- parent sees only linked student data
- total absence count is correct
- unexcused count is correct
- status badges are correct

19.8 Parent Submit Excuse

- parent can only submit excuses for linked child
- parent can only submit excuses for absent records
- duplicate excuse submission is blocked
- submitted excuse appears as pending
- notification is created after submission

19.9 Security

- RLS prevents parent from reading unrelated students
- RLS prevents teacher from reading unrelated classes
- RLS prevents teacher from editing unrelated attendance
- RLS prevents non-admins from managing system data
- service role key is never exposed to client

20. Build Order

Phase 1 — Project Setup

Build:

- Next.js app
- Tailwind setup
- Supabase client/server setup
- Vercel-ready environment variables
- base layout
- shared UI components

Output:

App runs locally
Can connect to Supabase
Basic UI shell exists

Phase 2 — Database + Auth

Build:

- migrations
- schema
- RLS policies
- seed data
- login page
- role-based redirects

Output:

Admin/teacher/parent demo users can log in
Each role redirects correctly
Protected routes work

Phase 3 — Admin Portal

Build:

- admin layout
- dashboard
- teachers page
- parents page
- students page
- classes page
- subjects page
- assignments page
- schedules page

Output:

Admin can manage the school's core data from the UI

Phase 4 — Teacher Attendance

Build:

- teacher layout
- attendance page
- class/session selector
- roster loading
- attendance initialization
- present/absent/late update
- summary counts

Output:

Teacher can take attendance for assigned class

Phase 5 — Teacher Lesson Logs

Build:

- lesson logs page
- class session selector
- lesson log form
- upsert behavior

Output:

Teacher can create/update lesson logs

Phase 6 — Parent Portal

Build:

- parent layout
- notifications page
- attendance page
- linked student selector if needed
- attendance summary/status logic

Output:

Parent can view student attendance and notifications

Phase 7 — Excuse Workflow

Build:

- parent submit excuse page
- teacher/admin excuse review
- status update logic
- notification generation

Output:

Parent can submit absence excuse

Teacher/admin can approve/reject

Parent receives updates

Phase 8 — Polish + Deployment

Build:

- loading states
- empty states
- error states
- mobile responsiveness
- seeded demo polish
- Vercel deployment

- production env vars

Output:

Production demo is usable end-to-end